

# Engineering Guidelines

---

Единый свод инженерных стандартов и практик команды.

Документ описывает базовые инженерные договорённости команды.

Цель — сделать разработку предсказуемой, код поддерживаемым, а процессы понятными для всех участников.

## Содержание

---

- [Обзор](#)
- [Release Management](#)
- [Git Flow](#)
- [Стратегия Merge](#)
- [Code Review](#)
- [История изменений](#)

## Обзор

---

Этот документ — точка входа для всех инженеров команды. Он описывает что мы делаем и почему, со ссылками на внешние стандарты и внутренние ADR с обоснованиями принятых решений.

---

## Release Management

---

Полный цикл [Release Management](#)

---

## Git Flow

---

В команде в качестве эксперимента на 3–4 спринта используется **классический Git Flow**.

Основные ветки:

- `master` — стабильная production-ветка;
- `release/*` — основная ветка разработки и ветка подготовки релиза;
- `feature/*` — ветки для новых задач;
- `bugfix/*` — исправления ошибок в рамках разработки или стабилизации релиза;
- `hotfix/*` — срочные исправления production.

Подробные правила работы с ветками описаны в [Git Flow](#).

Причины выбора стратегии, рассмотренные альтернативы, плюсы, минусы и критерии пересмотра описаны в [ADR: Выбор стратегии ветвления Git](#).

---

## Merge Strategy

---

1. **Sync:** Перед созданием ветки обнови `release/*` (`git pull origin release/*`).
2. **Branch:** Создай ветку `feature/....`
3. **Work:** Пишим код.
4. **Test:** Написать, проверить что работают(возможно договорится о каком то минимуме кроме happy path).
5. **Push & MR:** Merge Request.
6. **Review Fix:**
7. **Clean History (Squash & Amend)**
8. **Merge**

<details> <summary>Clean History (Squash & Amend)</summary>

## Что такое git commit --amend

---

Amend (Дополнить) - это способ переписать самый последний коммит.

- Сценарий: Ты сделал коммит, но забыл добавить один файл, или допустил опечатку в сообщении коммита.
- Правило: Можно делать только если ты еще не запустил коммит (или если это твоя личная ветка, и ты готов сделать force push).

Как сделать в IntelliJ IDEA:

- Открой окно Commit.
- Внеси нужные изменения.
- Поставь галочку Amend.
- Старое сообщение коммита подтянется автоматически. Исправь его если нужно.
- Нажми Commit.
- Если уже был пуш в ветку, то надо делать push force

## Что такое Squash

---

Squash - это объединение нескольких коммитов в один.

Как сделать в IntelliJ IDEA (Interactive Rebase):

Это самый мощный способ. Мы переписываем историю ветки локально перед тем, как отдать её на ревью или смирджить.

- Перейди на вкладку Git (внизу) -> Log.

- Найди в списке коммитов тот, от которого началась твоя ветка (например, последний коммит из develop).
- Нажми на него Правой Кнопкой Мыши -> Interactively Rebase from Here...
- Откроется окно:
- Оставь верхний коммит как Pick (это будет "голова" нового объединенного коммита).
- Все остальные коммиты ниже пометь как Squash (они вольются в верхний).
- Нажми Start Rebase.
- IDEA предложит объединить сообщения всех коммитов. Удали мусор, напиши одно красивое сообщение.

Squash через git:

- `git rebase develop`
- `git rebase -i develop`
- в файле все pick кроме верхнего поменять на s (squash)
- сохранить файл (в Vim это :wq и Enter).
- Сразу после этого откроется новое окно редактора. Git предложит тебе объединить сообщения всех коммитов.
- Удали лишний мусор и оставь одно красивое сообщение.
- Сохранить и закрыть файл.
- `git push --force` </details>

---

## Code Review

---

Все изменения в основные ветки проходят через Merge Request и Code Review.

Подробные правила подготовки MR, назначения reviewers, количества approve и обработки замечаний описаны в [Code Review Process](#).

---

## History Changes

---

| Дата       | Автор | Изменение                       |
|------------|-------|---------------------------------|
| 2026-MM-DD | Team  | Создан первый вариант документа |